Stolpe,
Graal-LSP

Graal-LSP: A Zero-overhead Polyglot Language Server Implementation for the GraalVM

Graal-LSP: Eine Overhead-freie Language Server Implementierung für die GraalVM

by

Daniel Stolpe

A thesis submitted to the
Hasso Plattner Institute
at the University of Potsdam, Germany
in partial fulfillment of the requirements for the degree of

Master of Science in IT-Systems Engineering

Supervisors

Prof. Dr. Robert Hirschfeld
Dr. John Doe

Software Architecture Group
Hasso Plattner Institute
University of Potsdam, Germany

December 26, 2018

Abstract

The english abstract.

Zusammenfassung

Die Zusammenfassung auf deutsch.

Acknowledgments

I owe everything to my cat.

Contents

1. Introduction **1**
1.1. Contributions . . . 1
1.2. Structure of this Thesis . . . 2

2. Context **3**
2.1. Terms and Concepts . . . 3
2.2. Truffle and GraalVM . . . 3
2.3. Language Server Protocol . . . 4

3. Approach **7**
3.1. A Language-agnostic Language Server . . . 7
3.1.1. Language Server Initialization . . . 8
3.2. Architecture + Language Server Bootstrapping . . . 11

4. Implementation **13**
4.1. Language Server Implementation . . . 13
4.1.1. Architecture . . . 13
4.1.2. Using runtime information . . . 13
4.1.3. LSP features implementation overview . . . 13
4.1.4. LSP client - VSCode extension example . . . 15
4.2. Requirements on Truffle Language Implementations . . . 15

5. Experience Report **17**
5.1. Between IDE and Language Workbench . . . 17
5.2. Completion response time . . . 17
5.3. Limitations . . . 17

6. Related Work **19**
6.1. Helvetia . . . 19
6.2. Self . . . 19
6.3. Smalltalk+Squeak . . . 19
6.4. Squimera . . . 19
6.5. Language Workbenches . . . 19
6.6. Run-time type collection . . . 19
6.7. CTags . . . 19

7. Conclusion and Future Work **21**

A. First Unimportant stuff. **25**

List of Figures

2.1. Truffle languages hosted by the Truffle framework on top of the GraalVM 5
2.2. example for how a tool and a language server communicate during a routine editing session 6

3.1. lang specific server vs lang-agnostic server 7

List of Tables

List of Listings

List of Abbreviations

LSP Language Server Protocol
IDE Integrated Development Environment
AST Abstract Syntax Tree
API Application Programming Interface
VM Virtual Machine
URI Uniform Resource Identifier

1. Introduction

- language development is challenging task
- what makes a good language: amongst performance, std lib, ecosystem and community, tools (for language developers and users) are important
- who provides the tools? language developers vs. language users
- normally tools are developed for a specific language and cannot be reused or need lots of adaptation
- In the Truffle framework, tooling (instrumentation) is part of the framework and therefore tools are implemented language-agnostic once
- Truffle has already some language-agnostic tools implemented (debugger, profiler, ...) and new ones can be easily built
- Truffle tooling support does not interfere with implementing high-performance languages
- Compared to a full IDE, some tools are missing
- Implementing LSP can provide further IDE-like features
- We propose an approach to build a generic LSP server on top of Truffle with:
  - minimal to zero language runtime performance overhead
  - minimal API changes to Truffle
  - minimal impl and maintenance overhead for language developers
- Benefits for the language developers: get IDE-like tools with minimal effort
- Benefits for language users: can choose from wide range of LSP clients + consistent dev-tools experience for different Truffle languages
- As proof of concept, we have implemented Graal-LSP- A Zero-overhead Polyglot Language Server Implementation for the GraalVM
- ?: focus on dynamically type languages? Or is this focus already included in Truffle (most impls are dynamically type langs)?

1.1. Contributions

- Contribution in detail:
  - An approach to build a generic LSP server on top of Truffle
  - An implementation of said approach: Graal-LSP
  - Proposals to extend the Truffle Framework with new interfaces needed to support more LSP features
  - An experience report how well existing Graal/Truffle-implemented languages work with Graal-LSP(with and without implementing the said interface proposals)

1.2. Structure of this Thesis

2. Context

In this chapter we explain terms, concepts and technologies helping to understand our approach in Chapter 3 and the implementation of Graal-LSP in Chapter 4.

2.1. Terms and Concepts

Abstract Syntax Tree (AST) Parsers for programming languages usually transform plain text source code into an intermediate representation which can be processed by compilers, interpreters or programming environment tools. An AST is such a typical representation which stores structural information of the source code implicitly in its tree structure. Therefore, concrete syntactic elements needed to write parsable code in plain text can be omitted. In general, inner nodes of the AST represent programming language operations and leaf nodes store the values needed for these operations. Depending on the use case, an AST can store additional information in its nodes. For example the original source section from which a node was derived. Programming environment tools can use these information to display any kind of errors, warnings or hints directly in the plain text source code. An AST interpreter evaluates an AST by traversing its tree structure and executing the operations defined in the nodes.

Scopes and Frames A Scope defines a region of a program in which a set of name bindings is valid, e.g. a local variable name. Most programming languages have a concept of a global (program wide) scope and local scopes defined by functions, blocks or modules. During execution of a program, frames store the actual values of the name bindings of visible scopes.

VM?

2.2. Truffle and GraalVM

Truffle is a language implementation framework written in Java [12]. The language developer uses the framework to write an AST interpreter, which is able to optimize itself during interpretation [15]. To achieve that, Truffle uses runtime type information to rewrite AST nodes. Callable elements of a language, e.g. functions or methods, are modeled as root nodes. They can be executed by providing a frame object. A node can have a source section attribute which encodes the range in the source code file it relates to. How source code is parsed and the AST is structured leaves Truffle in the responsibility of the language implementation.

Truffle has built-in support for developing language-agnostic tools by providing an instrumentation Application Programming Interface (API) [9]. It is designed to work well with the optimization techniques for language implementations so that instrumentation can be done with nearly zero-overhead [10]. Instruments can be installed by putting it on the Java class path and then be registered and unregistered dynamically as listeners for different events. For example, an instrument can be notified when a new source or source section is loaded, a statement is going to be executed or an expression will return a value. It is also possible to alter the current execution flow and let a function return a different value when called under certain conditions. This enables building powerful tools like debuggers or profilers which need to be implemented only once and can then be used for all Truffle languages. To control which AST nodes of a language are available for instrumentation, language implementors need to mark nodes as instrumentable. A tagging mechanism allows adding meta information, like "this node represents a statement in my language", which can be used by instruments to filter for specific nodes. For example, a debugger can facilitate these information to implement stepping. The tools available at the date of writing are mostly runtime-tools like debuggers, dynamic profilers and language extensions [10].

Cross-language interoperability is another feature of the Truffle framework [3]. Language implementations can be hosted by the Truffle framework in a multi-language runtime and communicate via language-independent messages. Foreign language objects can be accessed directly without marshalling.

To get high-performance language implementations, Truffle uses the language-independent just-in-time compiler named Graal. It has its roots in the research project of the Maxine VM [11]. Among other optimizations, Graal applies partial evaluation to compile stable AST subtrees into optimized machine code [13, 2]. These optimizations do not stop at language boundaries, so that the access of foreign language objects can also be optimized.

cite or footnote?

GraalVM[1]is the umbrella term for a HotSpot VM which is bundled with the Truffle language implementation and instrumentation framework and the Graal compiler, building a high-performance polyglot VM for applications written in a Truffle-implemented language [14]. Figure 2.1 visualizes that.

2.3. Language Server Protocol

In 2016 a group of IT companies, including Microsoft, Red Hat, and Codenvy, started to standardize the Language Server Protocol (LSP) [5]. This protocol describes an approach to tackle the problem of language and tool developers, which need to repeatedly implement language support (e.g. code completion, jump-to-definition and syntax error reporting) for every new code editor, Integrated Development Environment (IDE) or development tool. The LSP defines a *language server* and a *language client*. The server needs to be implemented only once for every language, providing the language-specific smarts. A development tool implementing a language client

[1] https://www.graalvm.org/

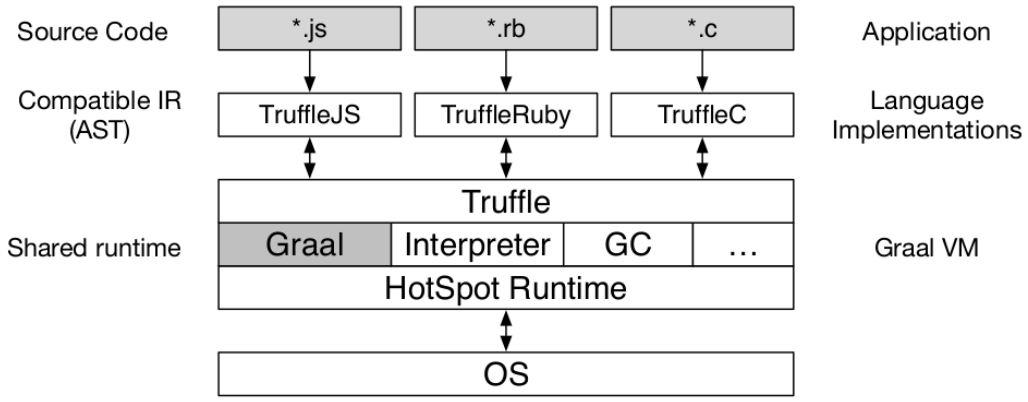


Figure 2.1.: Truffle languages hosted by the Truffle framework on top of the GraalVM

can make use of any language server and therefore support multiple languages with minimal effort.

At the date of writing, language servers for more than 50 languages exist[2], including almost all popular programming and scripting languages. Not all implementations support the full feature set defined in the LSP. Usually, these language server implementations support exactly one language, but the LSP allows multiple languages per server.

Language client and server communication LSP client and server communicate via JSON-RPC[3] requests, responses and notifications. The LSP version 3.7.0, which is the reference for our approach and implementation of Graal-LSP, specifies requests for typical features of an IDE: e.g. code and signature completion, navigational features (goto-definition, find all references and implementations, etc.), information on hover and code formatting. During initialization client and server can exchange which features are supported.

Figure 2.2 shows an exemplary communication between server and client. The client notifies the server that a file has been opened, the server parses the source code and pushes diagnostics to the client. Client-side changes to the source code are incrementally sent to the server.

LSP4J Our implementation of Graal-LSP uses LSP4J[4] version 0.4.x as Java binding for the Language Server Protocol. This framework maps the LSP requests and notifications to methods of Java interfaces so that the language server implementor does not have to deal with the JSON-RPC communication layer.

version

[2] https://microsoft.github.io/language-server-protocol/implementors/servers/ Sep. 2018
[3] https://www.jsonrpc.org/specification
[4] https://github.com/eclipse/lsp4j

2. *Context*

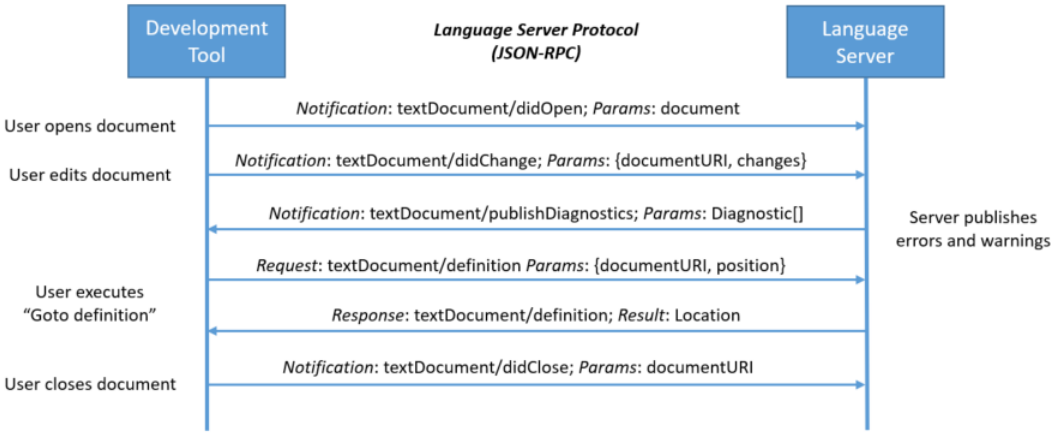


Figure 2.2.: example for how a tool and a language server communicate during a routine editing session

3. Approach

- language implementation + tool development = challenging task
- we propose: implement LSP for Truffle
- typically one specific language server impl for one language
- Our approach: a language-agnostic language server impl for languages created by the language implementation framework Truffle, Figure 3

3.1. A Language-agnostic Language Server

- LSP enables arbitrary language clients to communicate with arbitrary language servers
- Normally a language server provides support for one specific language only
- To build a language-agnostic language server for Truffle we have to define what Truffle languages need to provide in general to support LSP
- LSP is trade-off - there are lang-specific IDEs with more/other features
- Therefore our solution will also be a trade-off using general language features available for all Truffle languages

In the following sections we suggest how the existing Truffle language and instrumentation APIs can be facilitated to provide the needed data for LSP requests and notifications. When needed, we also suggest small API extensions to get access to additional language-specific information which have not been exposed yet, due to fact that the existing tools are mostly runtime-focused and do not require them.

Table lists and briefly explains all requests and notifications of the LSP version 3.7.0 which we cover in our approach. table

Covered LSP Features

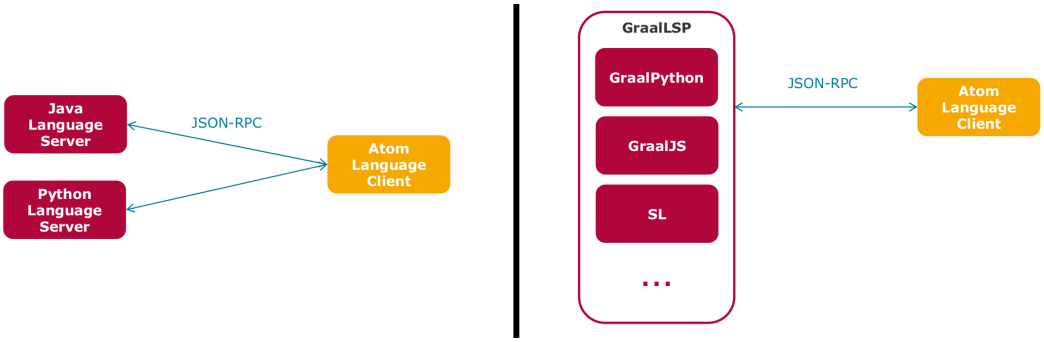


Figure 3.1.: lang specific server vs lang-agnostic server

- initialize
- diagnostics
- completion
- hover
- signatureHelp
- documentSymbol
- symbol
- definition
- references
- documentHighlight
- codeLens

3.1.1. Language Server Initialization

The first request sent from the client to the server is the "initialize" request. Among other parameters, it contains an Uniform Resource Identifier (URI) pointing to the root folder of the workspace and client capabilities, telling the server which LSP features the client supports. The server responds to this by sending server capabilities. When a server supports "completion" and "signatureHelp" the response needs to include trigger characters which tell the client when "completion" or "signatureHelp" shall be triggered automatically. A language-specific language server implementation can provide these characters statically. In our language-agnostic approach, where Truffle languages can be installed dynamically, we have to collect these characters for all available languages during server initialization. Using the Truffle instrumentation environment, we can get a list of all registered languages, but an interface extension is needed to get the trigger characters for "completion" and "signatureHelp".

Parsing Source Code

- a source is parsed when opened in the editor
- at start-up the files in the workspace are parsed
- collect all MIME-types of all languages
- walking file tree
- parsing every file which has a known MIME-type
- Client is notified about syntax errors via LSP Diagnostics

Diagnostics

- In our approach we use Diagnostics to notify the client about syntax errors in the source code. (Also used for highlighting code coverage, see ...)
- map syntax error to range in source file
- Truffle AST nodes have source sections
- Parsing-exceptions have a reference to erroneous node, if not -> use first char in first line
- Typically, exception is thrown on first syntax error
- => Display only first syntax error

Mapping the client caret position to AST node of a Truffle language

- Most LSP language features are context-sensitive and include line+column in the request
- Already explained: AST nodes have source sections
- The Truffle framework remembers every loaded Source and its source sections
- We use Truffle instrumentation to iterate over all loaded sections
- Precondition: Source needs to be parsable
- Two different use cases
- Find the "nearest" node: containing at end, next, previous, containing, root
- Find node at caret: containing
- Both filtered by node kind (Expression, Statement, Callable ...)

completion of globals/locals (static)

- language needs to derive a scope from an AST node
- if not parsable -> only global scope possible
- list valid symbols in the scope
- use TruffleObject interface to check object kind, documentation, signature, etc...
- for variables not many information can be display
- language-agnostic => Truffle has no type information for variables

Using runtime type information

- to display details e.g. about globals/locals, type information are needed
- easy for statically typed languages - not for dynamic
- so we need a generic approach
- idea: at runtime we can track type information (see type harvesting etc)
- -> learn
- run code
- challenges: side-effects, endless execution, (caret position not reached)
- side-effects -> sandboxing
- endless execution -> abort when caret position reached + TODO(timeout)
- caret position not reached -> define run-script-path to a script or test which covers the caret position
- run: observe frame at every source section, store a snapshot of the frame associated to the source section
- completion of globals/locals -> if node near caret has source section associated with coverage data -> use frame for globals/locals completion to get more details
- user interface to start code evaluation: see codeLense

completion of object properties

- define "object property completion"
- assumption: inserting a completion character => code might be not parsable
- fix code by undoing last change + parse again
- derive type information from node at caret

3. *Approach*

- run code to caret position
- optimizations: try different evaluation strategies
  - literal -> type can be derived without execution
  - coverage data (explained later)
  - run-to-section
  - globals
- getting infos from evaluation result object
- use TruffleObject interface to check object kind, documentation, signature, etc...

hover

- same preconditions as for completion (parsable code, find node at caret)
- if coverage data available: parse in-line + eval in-line
- for language developers: show debug infos (AST structure)

signatureHelp

- same preconditions as for completion (parsable code, find node at caret)
- need to know the signatureHelp trigger character
- when triggered -> assumption: code still parsable (e.g. editor has smart character insertion)
- find callable node at caret
- run to section
- listen for callable object
- ask via message to TruffleObject for signature and number of arguments

documentSymbol

- parsable code
- find all symbols in a specific source
- use Truffle Tagging -> new DefinitionTag needed to define which kind of symbol it is (class, variable, ...)
- fallback: search for RootTagged nodes

symbol

- find all symbols in workspace
- same as documentSymbol for all sources in workspace

documentHighlight

- parsable code + find node at caret
- variables only
- find other variable read/writes nodes in scope
- need to tag such nodes with ReadVariableTag+WriteVariableTag

definition

- parsable code + find node at caret
- function vs variable
- function
  - dynamic approach: run-to-section to get function object
  - ask function object for its source section of definition
  - static fallback: search for symbols in workspace
- variable
  - not all languages have a variable definition syntax -> first write is definition
  - so we interpret goto-definition for variables as goto-first-write
  - find first write in scopes

references

- parsable code + find node at caret
- function vs variable
- function
  - dynamic approach: run-to-section to get function object
  - search for function calls
- variable
- ...

Language features not implemented

- completion resolve
- typeDefinition
- implementation
- codeAction
- documentLink
- documentLink resolve
- documentColor
- colorPresentation
- formatting
- rangeFormatting
- onTypeFormatting
- rename
- foldingRange

3.2. Architecture + Language Server Bootstrapping

- graphic showing launcher, lsp server, truffle api communication
- comparison to other language server impls (built on top of language)

4. Implementation

4.1. Language Server Implementation

- lsp4j - Java binding for LSP (see Context)

4.1.1. Architecture

- Truffle-API + Graal-SDK
- Launcher, Instrument, Server
- lsp4j adapters

4.1.2. Using runtime information

Implementation of:

- Observe execution via instrumentation
- Side-effects -> Truffle-sandboxing
- Nested Contexts
- Evaluation strategies:
  - Literals
  - Coverage data
    - storing frames (copies)
    - source location format
    - migrating coverage data on source edit
  - Run-to-section
  - Globals

4.1.3. LSP features implementation overview

Feature impl matrix:

- TextDocumentService interface
  - completion
    - locals+globals vs object property completion (via checking completion character)
    - locals+globals completion
      - find nearest node

4. *Implementation*

    - check coverage data
    - globals only fallback
    - Used Truffle-API: findLocalScopes() + findGlobalScopes() + (parseInline() for coverage) + ForeignAccess.sendIsExecutable() + ForeignAccess.sendIsInstantiable() + ForeignAccess.sendIsNull()
    - Used new Truffle-API: env.boxPrimitive() + GetSignature + GetDocumentation

  * object property completion

    - fixing source code (undo last edit + re-parse)
    - find nearest node
    - tryDifferentEvalStrategies() (literals -> coverage data -> run-to-section -> globals)
    - Used Truffle-API: findLocalScopes() + findGlobalScopes()

  – hover
  – signatureHelp
  – definition
  – typeDefinition
  – implementation
  – references
  – documentHighlight
  – documentSymbol
  – codeAction
  – codeLense
  – formatting
  – rangeFormatting
  – onTypeFormatting
  – rename
  – didOpen
  – didChange
  – didClose
  – didSave
  – willSave
  – willSaveWaitUntil
  – documentLink
  – documentLinkResolve
  – documentColor
  – colorPresentation

- WorkspaceService interface

  – executeCommand
  – symbol
  – didChangeConfiguration
  – didChangeWatchedFiles

- didChangeWorkspaceFolders

4.1.4. LSP client - VSCode extension example

4.2. Requirements on Truffle Language Implementations

- Implement existing API:
  - ExpressionTag + StatementTag
  - findTopScopes()
  - findLocalScopes()
  - findMetaObject()
  - findSourceLocation()
  - parseInline()
- Implement proposed/new API:
  - getCompletionTriggerCharacters()
  - getSignatureHelpCharacters() (TODO)
  - boxPrimitive()
  - GetSignature + GetDocumentation custom message
  - DefinitionTag + ReadVariableTag + WriteVariableTag

5. Experience Report

5.1. Between IDE and Language Workbench

Language Workbench focus/aim: Efficient language (DSL) development, performance is not main focus vs. Graal/Truffle aim: Efficient execution

Assumption: Language Workbenches use static information only, our Truffle-based solution uses dynamic type information

5.2. Completion response time

5.3. Limitations

own chapter? or in 4?

- VSCode: Need to define supported languages in extension
- VSCode: Syntax highlighting defined at client-side
- LSP: No incremental sending of (completion) results
- Server: Only TCP (connecting to already running server), no std-in/-out impl
- Truffle: Only first syntax error is reported. Lang parsers fail on first error

6. Related Work

6.1. Helvetia

[8]

6.2. Self

6.3. Smalltalk+Squeak

6.4. Squimera

[7] [6]

6.5. Language Workbenches

[1]

6.6. Run-time type collection

[4]

6.7. CTags

- tagging a language object in an index file
- tag-file supported by editors
- allows user to locate the tagged object via name (goto-definition, list symbols)
- for new languages you need to define a regex or parser to find language objects
- only static analysis, no scoped completion (everything is suggested)

SemanticDB

7. Conclusion and Future Work

Bibliography

[1] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. "The State of the Art in Language Workbenches". In: *International Conference on Software Language Engineering*. Springer. Springer International Publishing, 2013, pp. 197–217. DOI: 10.1007/978-3-319-02654-1_11.

[2] Yoshihiko Futamura. "Partial evaluation of computation process–an approach to a compiler-compiler". In: *Higher-Order and Symbolic Computation* 12.4 (1999), pp. 381–391. DOI: 10.1023/A:1010095604496.

[3] Matthias Grimmer, Chris Seaton, Roland Schatz, Thomas Würthinger, and Hanspeter Mössenböck. "High-performance cross-language interoperability in a multi-language runtime". In: *ACM SIGPLAN Notices*. Vol. 51. 2. ACM. 2015, pp. 78–90. DOI: 10.1145/2936313.2816714.

[4] Michael Haupt, Michael Perscheid, and Robert Hirschfeld. "Type harvesting: a practical approach to obtaining typing information in dynamic programming languages". In: *Proceedings of the 2011 ACM Symposium on Applied Computing*. ACM. 2011, pp. 1282–1289. DOI: 10.1145/1982185.1982464.

[5] Microsoft. *Language Server Protocol*. https://microsoft.github.io/language-server-protocol/. Visited on 2018-09-24. 2017.

[6] Fabio Niephaus, Tim Felgentreff, Tobias Pape, and Robert Hirschfeld. "Squeak Makes a Good Python Debugger: Bringing Other Programming Languages Into Smalltalk's Tools". In: *Companion to the first International Conference on the Art, Science and Engineering of Programming*. ACM. 2017, p. 27. DOI: 10.1145/3079368.3079402.

[7] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. "Live Multi-language Development and Runtime Environments". In: *arXiv preprint arXiv:1803.10200* 2.3 (Mar. 2018). DOI: 10.22152/programming-journal.org/2018/2/8.

[8] Lukas Renggli, Tudor Gîrba, and Oscar Nierstrasz. "Embedding languages without breaking tools". In: *European Conference on Object-Oriented Programming*. Springer. 2010, pp. 380–404. DOI: 10.1007/978-3-642-14107-2_19.

[9] Michael L Van De Vanter. "Building debuggers and other tools: we can have it all". In: *Proceedings of the 10th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*. ACM. 2015, p. 2. DOI: 10.1145/2843915.2843917.

[10] Michael Van De Vanter, Chris Seaton, Michael Haupt, Christian Humer, and Thomas Würthinger. "Fast, Flexible, Polyglot Instrumentation Support for Debuggers and other Tools". In: *arXiv preprint arXiv:1803.10201* (2018). DOI: 10.22152/programming-journal.org/2018/2/14.

[11] Christian Wimmer, Michael Haupt, Michael L Van De Vanter, Mick Jordan, Laurent Daynès, and Douglas Simon. "Maxine: An approachable virtual machine for, and in, java". In: *ACM Transactions on Architecture and Code Optimization (TACO)* 9.4 (2013), p. 30. DOI: 10.1145/2400682.2400689.

[12] Christian Wimmer and Thomas Würthinger. "Truffle: a self-optimizing runtime system". In: *Proceedings of the 3rd annual conference on Systems, programming, and applications: software for humanity*. ACM. 2012, pp. 13–14. DOI: 10.1145/2384716.2384723.

[13] Thomas Würthinger, Christian Wimmer, Christian Humer, Andreas Wöß, Lukas Stadler, Chris Seaton, Gilles Duboscq, Doug Simon, and Matthias Grimmer. "Practical Partial Evaluation for High-performance Dynamic Language Runtimes". In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2017*. Vol. 52. 6. ACM. 2017, pp. 662–676. DOI: 10.1145/3062341.3062381.

[14] Thomas Würthinger, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Gregor Richards, Doug Simon, and Mario Wolczko. "One VM to rule them all". In: *Proceedings of the 2013 ACM international symposium on New ideas, new paradigms, and reflections on programming & software*. ACM. 2013, pp. 187–204. DOI: 10.1145/2509578.2509581.

[15] Thomas Würthinger, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Doug Simon, and Christian Wimmer. "Self-optimizing AST interpreters". In: *ACM SIGPLAN Notices*. Vol. 48. 2. ACM. 2012, pp. 73–82. DOI: 10.1145/2384577.2384587.

Appendix A.

First Unimportant stuff.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst sowie keine anderen Quellen und Hilfsmittel als die angegebenen benutzt habe.

Berlin, den 26. Dezember 2018

Daniel Stolpe

Todo list

cite or footnote? 4
version 5
table 7
own chapter? or in 4? 17